Local/cloud-switchable AI assistant with Graph tool-calling

LOCAL MICROSOFT 365 TRIAGE WORKSTATION

Overview

Built a text-based AI assistant that can run in local mode (Ollama) or cloud mode (OpenAI), with tool-calling access to the existing Microsoft Graph service layer, a new local codebase/doc search (RAG), and a UI-drivable propose/confirm/cancel workflow for any action that mutates live Microsoft 365 data. Reused the app's existing Graph capability gating, tool catalog, and RAG infrastructure rather than rebuilding them; the existing voice assistant (OpenAI Realtime), `/api/ask`, summarizer, portal parser, and TeamGPT integration were left untouched.

Files changed

New backend:

- `src/server/services/ai/modelProvider.js` — Ollama/OpenAI chat client + multi-round tool-calling loop (`runToolLoop`, `maxToolRounds` cap)
- `src/server/services/ai/embeddingProvider.js` — local/cloud embedding client, independent of chat mode. Reachability check matches the configured model against Ollama's `/v1/models` list tolerantly of the implicit `:latest` tag (a user's `nomic-embed-text` matches the listed `nomic-embed-text:latest`), so a pulled model isn't falsely reported unreachable. Same tag-tolerant fix applied to `modelProvider.js`'s local chat reachability check.
- `src/server/services/graph/assistantCapabilities.js` — friendly `mail.send` / `calendar.create` /etc. capability view, derived from the existing Graph catalog (no new scope list)
- `src/server/services/assistant/toolRegistry.js` — `list_graph_functions` + `run_graph_function` tool definitions (2-tool indirection over the ~100+ function catalog) + `search_docs` / `search_code`
- `src/server/services/assistant/pendingActions.js` — pending-action store interface (in-memory impl) + human-readable action summaries
- `src/server/services/assistant/controller.js` — chat orchestration, server-side tool-call validation, confirm/cancel, audit logging
- `src/server/services/codeKb/loader.js` — local codebase chunker (path exclusions, per-chunk secret scan)
- `src/server/services/codeKb/index.js` — codebase search service (same shape as `docsKb`)

New frontend:

- `frontend/src/lib/assistantChat.ts` — client for the new `/api/assistant/*` routes
- `frontend/src/components/assistant/AssistantChatPanel.tsx` — chat UI: capability badges, model-mode indicator, source chips, proposed-action cards with Confirm/Cancel

Modified:

- `src/server/config/env.js` — `ASSISTANT_MODEL_MODE` / `ASSISTANT_EMBEDDING_MODE` + local/cloud LLM and embedding env vars
- `src/server/app.js` — new routes: `GET /api/assistant/capabilities`, `GET /api/assistant/model-status`, `POST /api/assistant/chat`, `POST /api/assistant/actions/:id/confirm`, `POST /api/assistant/actions/:id/cancel`; `codeKb` added to `/api/health`
- `src/server/index.js` — wires up `codeKbService`, `assistantModelProvider`, `assistantPendingActionStore`, `assistantController`
- `src/server/services/docsKb/index.js`, `src/server/services/docsKb/vectorStore.js` — generalized to use the new embedding provider (model/client now parameters, not hardcoded OpenAI); cache key now includes the embedding model so switching providers invalidates stale vectors
- `frontend/src/components/assistant/RealtimeAssistantDock.tsx` — added a Voice/Text mode tab; voice UI is unchanged, Text mode renders `AssistantChatPanel`. Also fixed a pre-existing bug surfaced by testing this: the floating widget's position was clamped once (sized for the small collapsed icon) and never re-clamped on expand, so the much-larger expanded panel's right edge could render off-screen on narrower viewports — added a re-clamp `useEffect` keyed on `expanded`.

Change summary

- Local chat model: Ollama via its OpenAI-compatible Chat Completions endpoint (`LOCAL_LLM_BASE_URL`, default `http://localhost:11434/v1`). Cloud chat model: OpenAI (`CLOUD_LLM_PROVIDER=openai`, falls back to existing `OPENAI_API_KEY` / `OPENAI_MODEL` if `CLOUD_LLM_*` unset). No TeamGPT dependency in this assistant, per explicit instruction.
- Tool loop is server-orchestrated with a hard `maxToolRounds` cap (default 4): model → tool call → server validates + executes (read) or stages (write) → result fed back → repeat until final answer.
- Mutations are never executed directly by a tool call. `run_graph_function` on a mutating entry creates a pending action (human-readable summary, e.g. "Send email to X...") that only executes via `POST /api/assistant/actions/:id/confirm`; `cancel` discards it. Every propose/confirm/cancel/reject transition is appended to `.local-state/assistant-action-log.jsonl` (service/functionName/status only — never tokens or message content).
- Tool-call validation happens server-side regardless of what the model sends: unknown tool names, scopes no longer granted, and malformed arguments (via the existing `parseCatalogArgs`) are all rejected before anything reaches a Graph service call.
- System prompt includes an explicit rule that retrieved content (docs, code, email, Teams, tool output) is untrusted and must never be treated as instructions.
- Local codebase indexer chunks `src/server/` and `frontend/src/` (120-line windows), hard-excludes `node_modules`, `.git`, `.local-*`, `dist`, `build`, `graph-auths`, etc., and scans each chunk for likely-secret patterns (bearer tokens, private key blocks, connection strings, etc.) before embedding — matching chunks are dropped and logged by path, not indexed.

- Friendly capability view (`mail.read`, `mail.send`, ...) is derived entirely from the existing `graphTesterCatalog.js` + `graphCapabilities.js` (verified `functionName`s per capability) — no duplicate scope list to drift.

Validation

- `node --check` on all 13 touched/new backend files — pass.
- `npx tsc --noEmit` in `frontend/` — pass (one JSX fragment-wrapping bug found and fixed mid-session).
- `npx eslint` on the 3 touched/new frontend files — pass (one pre-existing, unrelated lint error at `RealtimeAssistantDock.tsx:982` left as-is, out of scope).
- `npm run build` in `frontend/` — production build succeeds.
- Live boot test (real cached Graph token, real `OPENAI_API_KEY`): `/api/health`, `/api/assistant/model-status`, `/api/assistant/capabilities`, `/api/assistant/chat` (plain + tool-calling), confirm/cancel on unknown IDs — all correct.
- Cloud mode: full tool-calling round trip verified end-to-end (list → run → grounded answer).
- Local mode (Ollama): tested 3 models empirically —
  - `qwen2.5-coder:7b` (Ollama-tagged "tools"-capable): does **not** reliably emit structured `tool_calls`; writes a tool-call-shaped JSON blob as plain text instead. Verified directly against Ollama's REST API, not just through this app's code.
  - `devstral-small-2:latest` (24B): correct format, but a multi-round tool loop took >5 minutes with no response on this hardware — too slow for interactive use.
  - `llama3.2:3b`: reliably emits correct `tool_calls` and responds quickly. Set as the default `LOCAL_LLM_MODEL`. Follows the "discover then call" instruction inconsistently (a 3B-model reasoning limitation) but every tool call it does make is validated server-side regardless, so a wrong/hallucinated function name is rejected before touching Graph, not silently acted on.
- Mutation propose → cancel → confirm-after-cancel → confirm-unknown-id: all four transitions verified against the real backend with a correctly-shaped `calendar.createEvent` call, with **no live Graph calls made** (deliberately not confirmed against the real calendar/mailbox — that's a real, visible action left for the user to trigger themselves).
- Missing-scope enforcement verified with a synthetic limited-scope set: `mail.send` capability correctly reports disabled with reason, and — more importantly — `sendMail` is genuinely absent from the `list_graph_functions` catalog output for that scope set, so the model cannot discover or call it (not just UI-hidden).
- Local-mode-unreachable behavior: initially returned an opaque `500 {"error":"Connection error."}`; fixed in `modelProvider.js` to catch connection errors and return a clear `502` with actionable guidance ("Confirm Ollama is running... ollama pull ..."), verified after the fix.
- Audit log (`.local-state/assistant-action-log.jsonl`) inspected directly — contains only `{timestamp, status, id, service, functionName, reason?}`, no tokens, no message/event content.

- `codeKb` / `search_code` exercised end-to-end through the running app: first call indexed 309 chunks from 114 files (`src/server/` + `frontend/src/`) and correctly answered "where is X implemented" with real file/line source references.
- Full browser verification (Playwright, headless Chromium, against `npm run dev` backend on :3000 + `npm --prefix frontend run dev` on :3011): loaded the dashboard, opened the Genny dock, switched to the new Text tab, confirmed all 20 capability badges and the model-mode indicator render, sent a plain message and got a real grounded reply, then drove a full mutation propose → Cancel flow and confirmed the "Confirm Calendar Update" / "Canceled — nothing was changed." states render correctly. Zero browser console errors throughout. Screenshots taken at each step.
  - This surfaced the pre-existing floating-panel positioning bug above (Text tab was initially unclickable — off-screen) — found and fixed before re-verifying.
  - `with_server.py` 's server cleanup left zombie `next dev` processes bound to ports 3000/3011 after failed runs (up to 1.2GB RSS) multiple times during testing; manually `taskkill /T /F` 'd the specific orphaned PIDs each time. Not an application bug — a test-harness process-cleanup gap on Windows with nested npm→next child processes.

Follow-up

- TeamGPT was not wired into this assistant's tool-calling path per explicit instruction; if cloud tool-calling via the internal Bedrock/Claude gateway is wanted later, `teamgpt/client.js` 's Bedrock Converse `toolConfig` support needs to be verified first.
- Pending-action store is in-memory only (by design, per the approved plan) — lost on server restart, not shared across instances. Swappable later behind the same interface if that becomes a problem.
- Local embeddings verified working end-to-end after the user ran `ollama pull nomic-embed-text` : with `ASSISTANT_EMBEDDING_MODE=local` + `ASSISTANT_MODEL_MODE=cloud` (the mixed mode the plan called for), a `docsKbService.search` produced correct 768-dim `nomic-embed-text` vectors and returned relevant results (calendar backend chunks for a calendar backend question). One-time cost: switching the embedding model invalidates the cache (cache key includes the model) and re-embeds — ~4.5 min for 337 doc chunks with the local model; cached afterward. `docs/.embeddings-cache.json` is a **tracked** git file; the test's local re-embed of it was restored to the committed OpenAI version afterward (`git restore`) so the tree isn't polluted with a 9.5MB diff / wrong-embedding-space cache.
- `qwen2.5-coder:7b` 's broken tool-calling was verified against this specific Ollama install/version; worth re-checking if Ollama or the model is upgraded.
- A real Graph mutation (actually confirming a proposed action, e.g. really sending a test email or creating a real calendar event) was intentionally never triggered in this session — that's a real, visible action against the user's live mailbox/calendar, left for the user to try themselves via the UI.